From Observability to Verifiability: An Execution Evidence Architecture for Agentic Software Systems

Anonymous Submission

March 2026

Abstract

Agentic software systems are increasingly expected to invoke tools, consume budgets, and produce externally consequential outputs, yet most current runtimes still expose execution primarily through logs and traces. Those artifacts are useful for observability, but they do not by themselves provide a bounded, reviewable, and independently checkable account of what was intended, what was authorized, what was executed, and what evidence was exported. This paper frames that mismatch as an execution evidence problem rather than a logging problem.

We present an execution evidence architecture for agentic software systems and instantiate it as a reproducible reference path in an artifact package omitted here for double-blind review. The architecture links five stages of runtime semantics: persona projection, intent/action/result objects, governance checkpointing, execution-integrity validation, and bounded audit receipts. The controlled evaluation directly tests the latter four evidence-preserving stages through a deterministic 16-task suite, a unified five-file export contract, a third-party bundle review script, minimal live CrewAI and LangChain baselines, paired same-framework comparisons in both frameworks with and without the evidence layer, four architectural ablations, and a small falsification suite for negative controls.

Across the checked-in evaluation artifacts, the full evidence-chain configuration improves average explicitness from 1.0 to 5.0, replayability from 3.0 to 5.0, tamper sensitivity from 0.0 to 4.81, audit boundedness from 2.0 to 5.0, and evidence-stage exposure from 1.0 to 5.0 relative to the local trace-oriented baseline. The minimal live CrewAI and LangChain baselines each reach replayability 4.0, but leave the other evidence dimensions at observability-like levels. In both same-framework paired comparisons, wrapping the identical live runtime with the evidence layer raises the corresponding evidence metrics to 5.0, 5.0, 4.81, 5.0, and 5.0. Ablation results show targeted degradation when intent capture, governance persistence, integrity validation, or receipt export is removed. Under an eight-scenario falsification suite, the updated review contract rejects cross-run replay mismatches, cross-bundle receipt swaps, receipt-digest forgeries, payload tampering, false tamper claims, and policy-bypass claims, yielding a 1.0 detection rate across all derived bundles and showing that the contract is not limited to field-presence checks. An appendix-only blinded review kit is included as workflow scaffolding, but not used as empirical evidence. In this controlled harness, the results support a bounded claim that observability surfaces and verifiability surfaces are not interchangeable.

Keywords. agentic software systems, execution evidence, audit receipts, verifiability, runtime governance

1 Introduction

Agentic software systems are increasingly used to execute software tasks, invoke tools, consume budgets, and produce outputs that matter beyond the runtime boundary. As these systems move from laboratory demos to operational workflows, the engineering question is no longer only

whether an agent can complete a task. It is also whether the resulting run can be independently reviewed, replayed, and trusted by a third party.

Current practice usually answers this need with observability artifacts such as logs, callback streams, trace records, and framework-specific execution histories. These artifacts are valuable for debugging and operational visibility, but they are not the same as execution evidence. A trace can show that something happened without making the requested action explicit, without preserving a governance decision, without exposing whether execution integrity was checked, and without exporting a bounded receipt that a reviewer can inspect without reconstructing the full runtime context.

This distinction matters because inspectability is weaker than verifiability. An operator may be able to browse a log and infer what likely happened, but an independent reviewer still lacks a stable artifact contract for answering four basic questions: was intent captured, was policy checked, was execution verified, and was a receipt exported? When those questions are left implicit, a run can be observable without being verifiable.

We refer to this mismatch as the *execution evidence gap*. The gap appears whenever a run claims that a task was requested, authorized, executed, and completed, yet the exported artifacts are insufficient to confirm those claims in a structured and reproducible way. In agent stacks this gap is easy to miss because observability tooling is often rich, while audit-oriented evidence surfaces remain ad hoc.

This paper presents an execution evidence architecture for agentic software systems. The proposal is not a new orchestration framework. Instead, it is a composable systems layer that links five stages of runtime semantics: persona projection, intent/action/result objects, governance checkpoints, execution-integrity validation, and bounded audit receipts. The purpose of the architecture is to make agent runs exportable as reviewable evidence bundles.

We instantiate the architecture as a reference path in an artifact package omitted here for double-blind review. The checked-in implementation includes a deterministic 16-task suite, a unified five-file export contract, a third-party review script, a trace-oriented baseline, minimal live CrewAI and LangChain baselines, paired same-framework comparisons in both runtimes with and without the evidence layer, four ablation modes, and a small falsification suite. An appendix-only blinded review kit is also included as reproducibility scaffolding. The resulting harness allows the paper to evaluate execution evidence as an implemented systems concern rather than as a purely conceptual abstraction.

The evaluation compares ten repository conditions: a local trace-oriented baseline, minimal live CrewAI and LangChain observability baselines, paired wrapped paths for those same two live runtimes, four evidence-chain ablations, and the full evidence-chain configuration. Across these settings we measure five dimensions: explicitness, replayability, tamper sensitivity, audit boundedness, and evidence-stage exposure. The checked-in summaries indicate that the full evidence chain improves every metric over the trace-oriented baseline, while the live CrewAI and LangChain baselines primarily improve replayability without recovering the missing evidence semantics. The paired live-runtime comparisons further show that the evidence layer changes the review surface even when the underlying framework path is held fixed. The architecture contains five stages, but the empirical analysis directly validates the latter four evidence-preserving stages while treating persona projection as an architectural precondition.

The contributions of this paper are threefold:

- It formulates execution evidence as a software systems problem that is distinct from runtime observability.
- It defines a five-stage execution evidence architecture, while directly evaluating the latter four evidence-preserving stages as an operational review contract.

- It implements and evaluates a reproducible reference path showing that evidence-oriented exports preserve review questions that trace-oriented exports do not explicitly retain.

The remainder of the paper is organized as follows. Section 2 introduces the method and reference architecture. Section 3 describes the evaluation setup. Section 4 reports the results across baselines and ablations. Section 5 positions the work against adjacent literature. Section 6 discusses limitations, boundary conditions, and implications for software engineering practice.

2 Method

2.1 Problem framing

The goal of this work is to define and evaluate a minimal execution evidence architecture for agentic software systems. The target problem is narrower than model correctness and broader than debugging. We do not attempt to prove that an agent's output is semantically correct, and we do not claim formal verification of arbitrary tool behavior. Instead, we ask a systems question: how should runtime artifacts be organized so that an independent reviewer can determine whether intent was captured, policy was checked, execution was verified, and evidence was exported?

This framing deliberately separates execution evidence from raw observability. Logs and traces remain useful, but they do not by themselves guarantee a stable review surface. The proposed method therefore centers on exported artifact contracts rather than hidden runtime state.

2.2 Execution evidence architecture

We define the architecture as a five-stage evidence chain:

1. persona projection,
2. intent/action/result objects,
3. governance checkpoint,
4. execution-integrity validation,
5. bounded audit receipt.

Persona projection. Persona projection provides the runtime-facing identity context of the acting system. In the current repository it functions as an entry-stage semantic anchor rather than as a separate evaluation target. The architecture needs a notion of who or what is acting, even when the downstream evidence metrics focus on later stages. If a deployment omits persona-level semantics, the remaining four stages still define the operational evidence contract evaluated here; persona projection is retained because accountability-oriented review still benefits from an explicit actor boundary.

Intent/action/result objects. Intent, action, and result objects convert a run from an implicit event sequence into structured semantics. The intent object captures the declared goal and expected artifact surface before execution. The action object records the operational step together with authorization-relevant context. The result object captures terminal state, including successful completion, policy blocking, and tamper-detected outcomes.

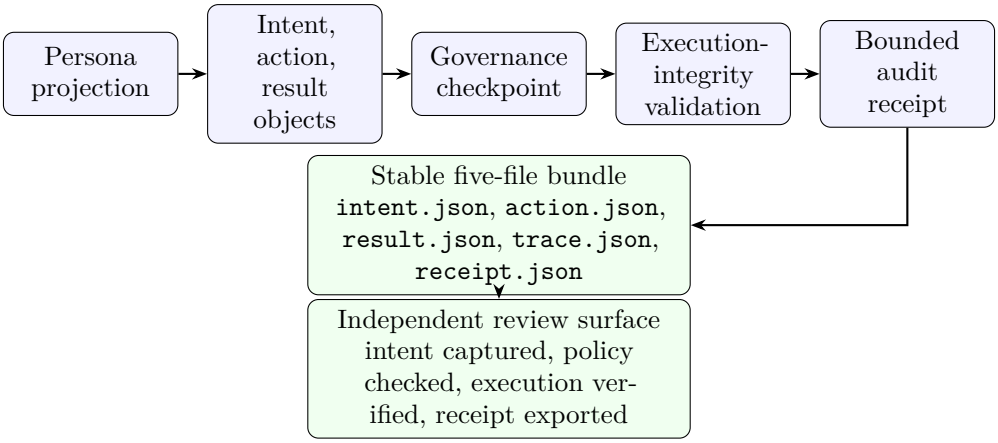

Figure 1: Execution evidence architecture as implemented in the repository. The five-stage chain produces a stable five-file bundle that can be consumed by an independent review script.

Governance checkpoint. The governance layer records whether a run was checked against policy. In the current harness this includes budget-aware and approval-aware blocking cases. The key design choice is that a blocked run is still evidence-bearing: policy denial is preserved as an explicit outcome instead of disappearing as missing output.

Execution-integrity validation. The integrity layer records whether the execution path was checked for replay-oriented consistency or tamper-sensitive mismatch. This layer is intentionally bounded. It does not provide a universal proof system; it provides enough structure to distinguish completed runs, non-verified runs, and tamper-detected runs within the artifact bundle.

Bounded audit receipt. The bounded receipt is the review-oriented export surface. Unlike a raw trace, it preserves only the fields needed to answer downstream audit questions. The receipt therefore compresses the run into a stable, third-party-facing representation.

Together these stages convert an agent run from framework-specific observability into a portable evidence bundle. Figure 1 summarizes the implemented path and the downstream review surface.

2.3 Reference path

The architecture is instantiated in an artifact package omitted here for double-blind review as a composable reference path. The implementation reuses concepts and naming from adjacent repositories, but keeps the evaluation logic centered in a deterministic local harness. This choice reduces hidden dependencies and makes every condition reproducible from checked-in task specifications and scripts.

For every task run, the harness exports exactly five JSON files:

- `intent.json`,
- `action.json`,
- `result.json`,
- `trace.json`,
- `receipt.json`.

These files are written under the stable layout `artifacts/runs/<task_id>/<mode>/`. The contract is shared across all conditions, including the baselines and ablations. This makes it possible to compare evidence shape directly without writing a framework-specific parser for each runtime condition.

2.4 Task suite

The evaluation harness includes 16 machine-readable tasks grouped into five categories:

- four read-only query tasks,
- four tool-call tasks,
- three budget-constrained tasks,
- three approval-required tasks,
- two tamper scenarios.

Each task specification records a task identifier, category, title, goal, input payload, expected artifacts, budget policy, approval policy, tamper case indicator, and notes. The mix of task classes is important because an evidence architecture should represent both positive and negative execution paths. In particular, the budget, approval, and tamper tasks test whether blocked and invalid runs appear as explicit outcomes rather than as absent output.

2.5 Experimental conditions

We evaluate ten repository conditions:

Baseline. The baseline is a local trace-oriented path with minimal evidence semantics. It approximates trace-only export without explicit intent capture, governance persistence, integrity validation, or bounded receipts.

Live framework baselines. The harness includes two minimal live framework baselines. `external_baseline` is a live CrewAI execution path driven by a deterministic local mock LLM. `langchain_baseline` is a live LangChain LCEL path driven by a deterministic local chat model. Both preserve framework-native execution evidence, but neither claims live end-to-end governance, integrity, or receipt support. The five exported files remain normalized by the paper harness so that the review contract stays comparable across conditions.

Same-framework wrapped paths. The paired same-framework conditions use the same live execution helpers as their corresponding baselines, but wrap those paths with explicit intent capture, governance persistence, integrity validation, and bounded receipt export. The CrewAI wrapped mode is named `external_evidence_chain`, while the LangChain wrapped mode is named `langchain_evidence_chain`. Their role is to isolate the effect of the evidence layer while keeping the live runtime fixed within each framework ecosystem.

Evidence chain. The evidence-chain condition is the full proposed architecture, including explicit intent capture, policy persistence, integrity validation, and bounded receipt export.

Ablations. The four ablations each remove one capability from the evidence chain: `no_intent`, `no_governance`, `no_integrity`, and `no_receipt`. These conditions isolate whether each stage contributes a distinct review function instead of merely adding redundant metadata.

2.6 Review logic and metrics

A third-party review script consumes a run directory and answers four yes/no questions:

1. was intent captured,
2. was policy checked,
3. was execution verified,
4. was a receipt exported.

The script emits machine-readable verdicts, an overall status, missing-field explanations, and a short summary. In the current revision it also recomputes bundle identity consistency and stored digests rather than trusting the exported status fields alone. This review step operationalizes the minimum independent inspection workflow that the architecture is intended to support.

We compare conditions using five deterministic, rule-based metrics derived from actual generated artifacts:

- **explicitness**: whether intent, policy, and outcome are directly inspectable;
- **replayability**: whether the run preserves enough structure for replay-oriented checking;
- **tamper sensitivity**: whether integrity-relevant violations appear as explicit outcomes;
- **audit boundedness**: whether the review surface is compact and purpose-oriented;
- **evidence-stage exposure**: how completely the harness exposes the intended evidence stages (repository field name: `integration_surface`).

Some parts of these metrics are directly measurable from the exported fields, while others are scored by transparent 0–5 rules documented in the repository. The method does not report semantic-quality scores or invented empirical values.

2.7 Appendix-only review scaffold

In addition to automated review, the artifact package includes a blinded human-review kit. The kit contains reviewer instructions, blinded bundle manifests, blank review sheets, timing guidance, and a summarizer for completed sheets. The checked-in outputs are synthetic smoke-test artifacts rather than a completed human-subject study, so the main paper does not use them as empirical evidence. Their role is methodological and appendix-only: they show that the bundle can be inspected by a reviewer workflow without requiring a web platform.

3 Evaluation Setup

3.1 Study goal

The evaluation asks whether execution evidence should be treated as a distinct systems layer rather than as an extension of runtime logging. Concretely, we test whether the full evidence

chain yields artifact bundles that are more explicit, more replayable, more tamper-sensitive, and more audit-ready than baseline observability surfaces.

3.2 Repository-grounded setup

All measurements in this draft come from the checked-in artifact package. The harness is exercised through the top-level make targets documented in the accompanying artifact notes. The evaluation sources are:

- the 16-task suite under `evaluation/tasks/`,
- the five-file export contract under `artifacts/runs/<task_id>/<mode>/`,
- comparison summaries under `artifacts/metrics/`,
- paper-support documentation under `docs/paper_support/`,
- the human-review kit under `evaluation/human_review/`.

The repository currently supports the following relevant workflows:

- `make paper-eval`,
- `make eval-external-baseline`,
- `make eval-framework-pair`,
- `make eval-langchain-pair`,
- `make eval-ablation`,
- `make falsification-checks`,
- `make top-journal-pack`.

3.3 Task distribution

The task suite is intentionally heterogeneous. Read-only and tool-call tasks cover ordinary successful runs. Budget-constrained and approval-required tasks exercise policy gating. Tamper scenarios exercise integrity-sensitive outcomes. Because the same 16 task specifications are used in every condition, differences in score are attributable to the exported evidence surface rather than to different workloads.

3.4 Conditions compared

The core automated comparisons are:

- **baseline** versus **evidence_chain**,
- **external_baseline** versus **evidence_chain**,
- **external_baseline** versus **external_evidence_chain** under the same live CrewAI path,
- **langchain_baseline** versus **langchain_evidence_chain** under the same live LangChain path,

Table 1: Worked bundle example from task-010, a budget-blocked run that remains independently reviewable.

File	Key field/value	Review role
intent.json	`intent.goal` = "Trigger a deterministic budget overflow . . . "	Captures pre-execution intent before any policy decision.
action.json	`authorization_state = blocked_budget`	Persists the governance verdict as an explicit authorization outcome.
result.json	`status = blocked_budget`	Records the terminal state as policy-blocked rather than missing.
trace.json	`integrity.verification_status = skipped_by_policy`	Distinguishes a reviewable non-execution outcome from a verified completion.
receipt.json	`questions_answered` maps intent, authorization, execution, and evidence	Exports a bounded review surface even though execution was blocked.

- a seven-mode comparison across baseline, external baseline, four ablations, and the evidence chain.

The main comparison asks whether the full architecture improves over a local trace-oriented export. The external baseline comparison asks whether a minimal live framework logging surface closes the same gap. The paired same-framework comparisons ask whether preserving the framework runtime while adding the evidence layer changes the review surface in more than one framework ecosystem. The ablation study asks whether removing any one stage causes interpretable degradation.

3.5 Outcome interpretation

The review script distinguishes among several outcome classes. A run may be marked *pass* when all four evidence questions are satisfied, *policy_blocked* when a budget or approval checkpoint blocks the task but still leaves reviewable evidence, *tamper_detected* when the integrity layer detects mismatch, or *partial* when one or more evidence-chain capabilities are missing.

This outcome model is important because it treats blocked and tampered runs as first-class results. In the full evidence chain, the artifact summaries show 11 completed and review-passing runs, 3 policy-blocked runs, and 2 tamper-detected runs across the 16-task suite. Table 1 gives a worked example from a policy-blocked bundle to show how a non-success outcome remains independently reviewable.

3.6 Falsification checks

To reduce the risk that the review contract merely accepts the repository's own positive cases, the artifact package also derives an eight-scenario falsification suite from previously reviewable bundles. Seven scenarios begin from the 11 evidence-chain bundles that the review script already marks as *pass*; the eighth (`policy_bypass_claim`) begins from the 3 evidence-chain bundles that originally reviewed as *policy_blocked*.

Single-surface removals such as missing intent and missing policy are rebound so that the mutated bundle stays internally consistent except for the targeted capability. Receipt forgery, payload tampering, cross-run replay mismatch, and cross-bundle receipt swap exercise strengthened bundle-identity and digest-binding checks rather than trusting stored status fields. The policy-bypass claim scenario exercises cross-file coherence directly by rewriting a blocked bundle to claim successful completion while leaving the persisted blocked policy decision in place. These checks are reported separately from the main baseline and ablation comparisons because they

Table 2: Trace-oriented baseline versus full evidence chain. Counts report positive bundle-review verdicts across the 16-task suite; scores are average rule-based metrics on a 0–5 scale. The evidence-chain condition includes 11 verified successful executions, 3 policy-blocked reviewable outcomes, and 2 tamper-detected reviewable outcomes.

Mode	Intent	Policy	Exec. verified	Receipt	Explicitness	Replayability	Tamper	Audit	Stage exposure
Baseline	0/16	0/16	0/16	0/16	1.0	3.0	0.0	2.0	1.0
Evidence chain	16/16	16/16	11/16	16/16	5.0	5.0	4.81	5.0	5.0

do not measure runtime quality; they measure whether the independent review contract can reject internally contradictory or transplanted artifacts.

3.7 Appendix-only reviewer workflow

The artifact package also includes a blinded reviewer workflow, but the current manuscript does not treat it as part of the main empirical evaluation. Its checked-in outputs are synthetic smoke-test artifacts rather than human-subject results, so we confine it to the appendix and use it only as reproducibility scaffolding.

4 Results

4.1 Baseline versus full evidence chain

Table 2 reports the main baseline comparison. The full evidence chain improves every measured dimension over the trace-oriented baseline. Intent capture, policy checking, and receipt export all move from zero observed positive reviews in the baseline to 16 of 16 tasks in the evidence chain. The repository metric records *execution verified* as 11 of 16, but that count should be read in two layers: among the 13 runs that reached the execution stage, 11 completed with verification success and 2 ended as explicit tamper-detected outcomes; the remaining 3 tasks are reviewable policy-blocked outcomes rather than missing bundles.

The metric changes are large. Average explicitness rises from 1.0 to 5.0, replayability from 3.0 to 5.0, tamper sensitivity from 0.0 to 4.81, audit boundedness from 2.0 to 5.0, and evidence-stage exposure from 1.0 to 5.0. This pattern supports the paper's narrower claim: within a stable exported-bundle contract, the evidence-chain surface preserves review questions that the trace-oriented surface does not explicitly retain.

4.2 External baseline comparison

Table 3 compares the minimal live CrewAI baseline with the full evidence chain. The external baseline improves replayability relative to the local baseline, reaching an average score of 4.0, because a real CrewAI runtime now produces the task execution surface that is later normalized into the five-file bundle. However, it still leaves explicit intent capture, policy persistence, integrity verification, and receipt export absent. As a result, explicitness, tamper sensitivity, audit boundedness, and evidence-stage exposure remain at 1.0, 0.0, 2.0, and 1.0 respectively.

This comparison only partially addresses the straw-baseline concern. Because the baseline is a minimal live integration rather than a full production benchmark, the result should be read as evidence that a framework-native default logging surface still leaves major evidence gaps by default, not as a definitive benchmark against every production framework.

Table 3: Framework-shaped external baseline versus the full evidence chain. The external baseline preserves a richer default execution surface than the local trace baseline, but does not add explicit evidence-chain semantics.

Mode	Intent	Policy	Exec. verified	Receipt	Explicitness	Replayability	Tamper	Audit	Stage exposure
External baseline	0/16	0/16	0/16	0/16	1.0	4.0	0.0	2.0	1.0
Evidence chain	16/16	16/16	11/16	16/16	5.0	5.0	4.81	5.0	5.0

Table 4: Same-framework paired comparisons under the live CrewAI and LangChain paths. In both frameworks the baseline preserves only the default framework-shaped observability surface, while the paired condition wraps that same runtime with the full evidence chain.

Framework	Mode	Intent	Policy	Exec. verified	Receipt	Explicitness	Replayability	Tamper	Audit	Stage exposure
CrewAI	CrewAI baseline	0/16	0/16	0/16	0/16	1.0	4.0	0.0	2.0	1.0
CrewAI	CrewAI + evidence chain	16/16	16/16	11/16	16/16	5.0	5.0	4.81	5.0	5.0
LangChain	LangChain baseline	0/16	0/16	0/16	0/16	1.0	4.0	0.0	2.0	1.0
LangChain	LangChain + evidence chain	16/16	16/16	11/16	16/16	5.0	5.0	4.81	5.0	5.0

4.3 Same-framework paired comparisons

Table 4 tightens the external-baseline argument by keeping the live runtime fixed inside two framework ecosystems. The repository now reports one paired comparison for CrewAI and one for LangChain. In each case, the baseline preserves only the default framework-shaped observability surface, while the paired condition wraps that same live execution path with explicit intent capture, governance persistence, integrity validation, and bounded receipt export.

The paired result is consistent across both frameworks. The default CrewAI and LangChain baselines each remain at 1.0 explicitness, 4.0 replayability, 0.0 tamper sensitivity, 2.0 audit boundedness, and 1.0 evidence-stage exposure. The wrapped conditions in both frameworks rise to 5.0, 5.0, 4.81, 5.0, and 5.0 while preserving the same live runtime path within their respective ecosystems. This does not eliminate all external-validity concerns, because both integrations remain minimal and local, but it materially reduces the claim that the paper only wins by comparing a custom local harness against a single weaker framework proxy.

4.4 Ablation results

Table 5 summarizes the seven-mode comparison. The ablations show targeted degradation rather than uniform collapse.

Removing intent capture lowers explicitness from 5.0 to 3.0 and evidence-stage exposure from 5.0 to 4.0. Under the strengthened review contract it also clears all positive receipt-export verdicts, because the bounded receipt can no longer answer all four review questions once the intent surface is removed. Removing governance lowers explicitness to 2.0, eliminates all positive policy-checked reviews, and changes the task suite behavior by allowing formerly blocked tasks to complete, while tamper sensitivity remains high because integrity checks are still present. Positive receipt-review verdicts also drop to 0 of 16 in that mode, because the bundle can no longer substantiate what was authorized. Removing integrity leaves explicitness and audit boundedness at evidence-chain levels, but drives tamper sensitivity and execution verification to 0.0 and 0 of 16. Removing receipt export preserves most other capabilities but drops audit boundedness from 5.0 to 2.0 and receipt-exported reviews from 16 of 16 to 0 of 16.

Taken together, the ablations indicate that each stage contributes a distinct review function. The evidence chain therefore behaves as a layered architecture rather than as a single monolithic switch.

Table 5: Seven-mode comparison across the baseline, external baseline, four ablations, and the full evidence chain. Scores are average 0–5 values; counts report positive review verdicts across 16 tasks.

Mode	Explicitness	Replayability	Tamper	Audit	Stage exposure	Intent	Policy	Exec. verified	Receipt
Baseline	1.0	3.0	0.0	2.0	1.0	0/16	0/16	0/16	0/16
External baseline	1.0	4.0	0.0	2.0	1.0	0/16	0/16	0/16	0/16
No intent	3.0	4.0	4.81	4.0	4.0	0/16	16/16	11/16	0/16
No governance	2.0	5.0	5.0	4.0	4.0	16/16	0/16	14/16	0/16
No integrity	5.0	5.0	0.0	5.0	4.0	16/16	16/16	0/16	16/16
No receipt	4.0	4.0	4.81	2.0	4.0	16/16	16/16	11/16	0/16
Evidence chain	5.0	5.0	4.81	5.0	5.0	16/16	16/16	11/16	16/16

Table 6: Negative-control bundle checks derived from review-passing and policy-blocked evidence-chain runs. Each scenario is re-reviewed with the same independent bundle contract used in the main study.

Scenario	Source	Targeted failure	Detected	Intent	Policy	Exec. verified	Receipt
Missing intent	Pass	Intent	11/11	0/11	11/11	11/11	11/11
Missing policy	Pass	Policy	11/11	11/11	0/11	11/11	11/11
Forged receipt	Pass	Receipt	11/11	11/11	11/11	11/11	0/11
Payload tamper	Pass	Execution + receipt	11/11	11/11	11/11	0/11	0/11
False tamper claim	Pass	Tamper claim	11/11	11/11	11/11	0/11	11/11
Cross-run replay mismatch	Pass	Replay chain	11/11	11/11	11/11	0/11	11/11
Cross-bundle receipt swap	Pass	Receipt binding	11/11	11/11	11/11	11/11	0/11
Policy bypass claim	Policy-blocked	Policy/result coherence	3/3	3/3	3/3	3/3	3/3

4.5 Falsification resistance

Table 6 reports the strengthened negative-control suite. Seven scenarios are derived from the 11 evidence-chain bundles that originally reviewed as *pass*, and one scenario is derived from the 3 evidence-chain bundles that originally reviewed as *policy_blocked*. Every derived bundle is downgraded to *partial*, yielding a 1.0 detection rate across all eight falsification scenarios.

The pattern of failure is selective rather than indiscriminate. Removing the intent object clears only the intent verdict while leaving policy, execution verification, and receipt export positive in all 11 pass-derived bundles. Removing the persisted policy decision clears only the policy verdict. Forging a receipt digest and swapping in a receipt from another bundle both clear only the receipt verdict, showing that receipt binding is checked independently of execution verification. Tampering with the result payload after export breaks execution verification and receipt verification together because the recomputed digests no longer match the stored bundle. Replacing the trace with one from another run clears execution verification while leaving the receipt correctly rebound to the mutated bundle, which separates replay-chain integrity from receipt binding. Declaring a tamper outcome without a corroborating digest mismatch is also rejected in all 11 cases, showing that the review script does not merely trust a `tamper_detected` label.

The policy-bypass claim scenario is the strongest new check. In all 3 source bundles, intent capture, policy persistence, execution verification, and receipt export still remain individually positive after falsification, but the bundle is nonetheless rejected because the claimed completed outcome contradicts the persisted blocked policy decision. This shows that the review contract is no longer limited to field presence and digest recomputation; it now also rejects cross-file contradictions that would otherwise masquerade as valid evidence.

These checks do not solve the external-benchmark problem, but they do tighten one internal-validity concern. The repository is no longer only scoring architecture-aligned positives against

architecture-aligned metrics; it also demonstrates that the same review contract rejects malformed, transplanted, or over-claimed bundles derived from previously valid runs.

4.6 Appendix-only reviewer note

The artifact package includes a blinded reviewer workflow, but its checked-in outputs are synthetic smoke-test artifacts rather than human-subject results. We therefore exclude it from the main result claims and retain it only as appendix-level protocol scaffolding; the details appear in Appendix A.2.

5 Related Work

This work sits at the intersection of runtime observability, provenance and audit modeling, policy enforcement, and agent-framework execution tooling. The paper does not attempt to replace those lines of work. Instead, it argues that agentic software systems need an additional bridge layer that turns runtime traces into bounded evidence bundles for independent review.

Runtime observability systems standardize logs, metrics, and traces as the primary surface for inspection. OpenTelemetry is representative of this design center [6], while framework-specific tooling such as LangSmith observability and CrewAI tracing exposes agent traces, callbacks, and task timelines for debugging and operations [2, 3]. These systems make runs inspectable, but they do not by default provide a review contract that explicitly preserves intent capture, policy persistence, integrity status, and receipt export.

Provenance research emphasizes lineage, responsibility, and reproducibility across heterogeneous systems. PROV-O remains a foundational provenance vocabulary for interoperable provenance interchange [4], and recent work such as PROV-AGENT extends this tradition to agentic workflows by capturing prompts, responses, and decisions as workflow provenance [7]. Our execution-evidence layer is narrower than full provenance graphs: it compresses the run into a bounded bundle designed specifically for third-party review questions.

Policy systems show that authorization can be externalized and made explicit. Open Policy Agent is a widely used example of policy-as-code infrastructure [5]. Recent receipt-oriented work also argues that lightweight execution receipts can offer a practical middle ground between unstructured logs and heavyweight cryptographic proof systems for interactive agents [1]. The present paper adopts that general intuition, but evaluates it at the level of exported run bundles, review logic, and controlled architectural ablations.

Relative to these strands, the claim of this paper is modest. Observability surfaces, provenance models, and policy engines each solve part of the problem, but they do not automatically yield a bounded review bundle that a third party can inspect without reconstructing the original runtime context. The operational novelty claimed here is therefore not telemetry, provenance, or policy evaluation in isolation; it is the combination of a stable five-file review bundle, a four-question independent review contract, and a controlled ablation study that separates the stages. The proposed architecture is intended as that bridge, not as a replacement for tracing, provenance, or policy tooling.

6 Discussion and Threats to Validity

6.1 What the results support

The repository results support a bounded systems claim: execution evidence can be modeled as a distinct layer above runtime observability and below downstream audit interpretation. In the implemented harness, this layer improves pre-execution explicitness, preserves governance outcomes as evidence-bearing states, detects tamper scenarios as explicit review results, exports bounded receipts that a third party can inspect without reading a full trace transcript, and rejects several falsified bundle variants derived from previously valid runs.

6.2 What the results do not support

The results do not show semantic correctness of model outputs. They do not provide formal verification of arbitrary agent behavior. They do not establish that the current external baseline is a complete representation of any production framework, and they do not yet provide a real human-subject study. The paper should therefore be read as a reference-architecture and reference-implementation study, not as a claim of universal runtime coverage.

6.3 Threats to validity

Construct validity. The metrics are evidence-shape metrics rather than semantic task-quality metrics. This is deliberate, because the paper studies reviewability and verifiability surfaces. Still, the conclusions should be limited to those properties.

Internal validity. The harness is deterministic and local. That improves reproducibility, but it also means that the system does not exercise the nondeterminism, service failures, and integration drift of production agent stacks.

External validity. The live framework baselines are minimal local integrations, not full production benchmarks. The paired comparisons now cover both CrewAI and LangChain, which strengthens the paper by showing that the evidence layer changes the review surface even when the live runtime is held fixed in two framework ecosystems. That still does not justify generalization to all framework-native logging surfaces, all production CrewAI deployments, or all LangChain-based systems.

Appendix-only reviewer workflow. The current blinded reviewer workflow is a kit plus a synthetic smoke test. It demonstrates process reproducibility, but it does not support claims about reviewer performance, statistical significance, or organizational adoption. For that reason, the main paper treats it as appendix material rather than as part of the empirical result core.

Adversarial validity. The current revision is stronger than a pure positive-only harness because it now includes repository-defined falsification checks for omission, replay mismatch, receipt swap, policy-bypass claims, payload tampering, and false tamper claims against previously valid bundles. The review layer therefore fails on more than missing fields: it also fails on contradictory policy/result states, transplanted bundle fragments, and cross-file coherence violations. Remaining gaps include hidden-field omission beyond the current five-file contract, receipt-forgery strategies that evade the present bounded digest checks, open-ended adversarial

campaigns authored outside the repository, and live framework falsification under nondeterministic services. Future work should target those broader attack surfaces rather than treating the current suite as exhaustive.

Architectural scope of persona projection. Persona projection is the least validated stage in the current implementation. It is retained because a review bundle still needs an actor boundary, but the present experiments do not establish an independent causal effect of persona semantics. For that reason, the paper's operational claim is intentionally narrower than the full five-stage architecture: the controlled evidence evaluation directly supports the latter four stages.

6.4 Implications for software engineering

Despite these limits, the reference path is useful. It suggests that software engineering for agentic systems should distinguish between *runtime telemetry*, which helps operators watch a system, and *execution evidence*, which helps reviewers verify what the system claimed to do. That distinction matters in settings where tool use, budgets, approvals, and external outputs need a compact evidence trail rather than an unbounded stream of internal events.

7 Conclusion

This paper presented an execution evidence architecture for agentic software systems and instantiated it as a reproducible reference path in an artifact package omitted here for double-blind review. The central claim is that observability alone is not enough: agent runs need structured evidence bundles that make intent, authorization, execution verification, and exported receipts independently reviewable.

The checked-in evaluation harness supports a bounded version of that claim with a deterministic 16-task suite, a stable five-file export contract, a third-party review script, minimal live CrewAI and LangChain baselines, paired same-framework comparisons in both frameworks with and without the evidence layer, four ablation modes, and a small negative-control falsification suite. An appendix-only blinded reviewer workflow is also included as reproducibility scaffolding. Across the current summaries, the full evidence chain preserves review questions that the trace-oriented baseline and the live framework observability baselines do not explicitly retain, the same live runtime becomes reviewable when wrapped by the evidence layer in two framework ecosystems, the architecture shows interpretable degradation under targeted ablations, and the same review contract rejects several malformed or over-claimed bundles derived from previously valid runs.

Future work should validate the architecture against live framework integrations, extend the integrity layer beyond the current bounded checks, and run a real blinded reviewer study. Even in its current form, the reference implementation shows a plausible and reproducible path for engineering execution evidence as a first-class systems layer rather than treating it as a side effect of logging.

A Reproduction Notes

The manuscript is grounded in the checked-in artifact package rather than in a separate private experiment log. The relevant source material includes:

- `docs/paper_support/task-suite.md`,

- `docs/paper_support/export-format.md`,
- `docs/paper_support/review-workflow.md`,
- `docs/paper_support/comparison-workflow.md`,
- `docs/paper_support/external-baseline.md`,
- `docs/paper_support/same-framework-comparison.md`,
- `docs/paper_support/langchain-comparison.md`,
- `docs/paper_support/ablation-study.md`,
- `docs/paper_support/falsification-workflow.md`,
- `docs/paper_support/falsification-summary.md`,
- `docs/paper_support/human-review-study.md`,
- `artifacts/metrics/comparison-summary.json`,
- `artifacts/metrics/external-baseline-summary.json`,
- `artifacts/metrics/framework-pair-summary.json`,
- `artifacts/metrics/langchain-pair-summary.json`,
- `artifacts/metrics/ablation-summary.json`,
- `artifacts/metrics/falsification-summary.json`,
- `artifacts/human_review/synthetic-review-summary.json`.

Within the manuscript directory, `python3` and the helper script `scripts/generate_tables.py` regenerate the checked-in table fragments from the JSON metric summaries. The falsification suite is refreshed from the repository root via `make falsification-checks`, which updates the checked-in falsification narrative and JSON summary artifacts before the manuscript tables are rebuilt. The `make` target then builds the PDF through `latexmk`. The current bibliography is intentionally minimal rather than exhaustive; it is limited to verified standards, official framework documentation, and primary papers used to ground the present draft.

The checked-in live-framework artifacts were generated from the repository-local virtual environment. In the current revision, the paired CrewAI comparison uses `crewai 1.10.1`, and the paired LangChain comparison uses `langchain 1.2.12`.

A.1 Worked bundle example

Table 1 illustrates the bundle surface for a policy-blocked run. The example shows why a non-success outcome can still be independently reviewable: intent is captured before execution, policy denial is persisted as an authorization state, the result records a blocked terminal status, the trace marks verification as skipped by policy rather than silently absent, and the receipt still exports a bounded digest set.

A.2 Appendix-only reviewer workflow note

The checked-in human-review sheet is synthetic and is included only to verify that the blinded bundle workflow, answer key, and summarizer operate end to end. The synthetic summary covers eight blinded bundle rows, split evenly between baseline and evidence-chain conditions. Both conditions reach question accuracy 0.9375 and mean confidence 3.5; mean review time is 117.5 seconds for baseline bundles and 161.5 seconds for evidence-chain bundles. These values are not used in the paper as usability evidence, are not part of the main results, and should not be interpreted as a real reviewer study.

References

[1] Abhinaba Basu. Tool receipts, not zero-knowledge proofs: Practical hallucination detection for ai agents. https://arxiv.org/abs/2603.10060, 2026. Accessed: 2026-03-13.

[2] CrewAI. Crewai tracing. https://docs.crewai.com/en/observability/tracing, 2026. Accessed: 2026-03-13.

[3] LangChain. Langsmith observability. https://docs.langchain.com/langsmith/observability, 2026. Accessed: 2026-03-13.

[4] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. https://www.w3.org/TR/prov-o/, 2013. W3C Recommendation.

[5] Open Policy Agent Authors. Open policy agent documentation. https://www.openpolicyagent.org/docs, 2026. Accessed: 2026-03-13.

[6] OpenTelemetry Authors. What is opentelemetry? https://opentelemetry.io/docs/what-is-opentelemetry/, 2026. Accessed: 2026-03-13.

[7] Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira da Silva. PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows. In *2025 IEEE International Conference on eScience (eScience)*, 2025. doi: 10.1109/eScience65000.2025.00093. URL https://arxiv.org/abs/2508.02866.